DEVELOPMENT OF AN IMAGE-BASED GUI SYSTEM FOR REAL TIME DETECTION AND CLASSIFICATION OF FRUITS.

BY

IZOKPU ELIZABETH EHIMEN

17/ENG02/088

A REPORT SUBMITTED TO THE

DEPARTMENT OF ELECTRICAL, ELECTRONICS AND COMPUTER ENGINEERING

AFE BABALOLA UNIVERSITY, ADO-EKITI

EKITI STATE, NIGERIA

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF

BACHELOR OF ENGINEERING

July, 2021

CERTIFICATION

This is to certify that **IZOKPU ELIZABETH EHIMEN** with matriculation number **17/ENG02/088** carried out this research work under my supervision, in partial fulfillment for the award of Bachelor of Engineering in the Department of Electrical, Electronics and Computer Engineering, Afe Babalola University, Ado-Ekiti.

Engr. Adeniji Kehinde _____

Project Supervisor **Signature and Date**

Dr Onibonoje Femi _____

Head of Department **Signature and Date**

DEDICATION

I dedicate this report to God Almighty in which through him all things were made possible. I would also like to dedicate this report to my parents, Mr. and Mrs. Izokpu, my siblings and friends.

ACKNOWLEDGEMENT

I have put efforts in this project. However, it would not have been possible without Almighty God for giving me the essence of life and strength to pull through. A special thanks to my wonderful parents Mr. Francis Anetor Izokpu and Mrs. Mary Ehisunoria Izokpu for their continuous support and words of encouragement throughout the process.

I would also like to extend my sincere thanks to my beautiful friends for their kind co-operation and continuous support in the completion of this project.

I am highly indebted to my supervisor Engr. Kehinde Adeniji for his guidance and constant supervision as well as for providing necessary information regarding the project & also for their support in completing the project.

ABSTRACT

This work presents an image-based real-time system to detect and classify fruits. Fruit classification can be used to perform the sorting and classification process automatically. A traditional method for fruit classification is manual sorting which is time consuming and involves human presence always.

First, extraction of certain features from fruit images- watermelon, apple, banana were used. Segmentation, thresholding and application of machine learning were used to train and classify these fruits on MATLAB.A wireless connection was set up between the PC device and IP camera which serves as means of taking input to the developed application.

A Graphical User Interface is designed on MATLAB to implement this model via the IP camera that the machine learning algorithm will use to detect and classify the fruits.

TABLE OF CONTENTS

	Pages
Certification	ii
Dedication	iii
Acknowledgement	iv
Abstract	v
Table of Contents	vi-vii
List of Figures	viii
CHAPTER ONE: INTRODUCTION	
1.1 Background of Study	1-2
1.2 Statement of Problem	2
1.3 Aim and Objectives	2
1.4 Justification of Study	3
1.5 Scope of Study	3
1.6 Organization of Report	3
CHAPTER TWO: LITERATURE REVIEW	
2.1 Machine Vision	4
2.2 Image Processing	4-5
2.3 How Image Processing works	5
2.4 Overview of Template Matching	6-9
2.5 Related Works	10-14
CHAPTER THREE: METHODOLOGY	
3.1 System Requirement	15

3.1.1 Hardware Requirements 15

3.1.2 Software Requirements 16

3.1.3 Functional and Non-Functional Requirements 16

3.2 System Design 16

3.2.1 Block Diagram of the system 17-24

3.2.2 Flow Chart of the system 25

3.2.3 Graphical User Interface of the system 26

3.3 Data Processing 26

3.3.1 Data Collection 26

3.3.2 Data Pre-processing 27

3.4 Implementation of the algorithm 27-28

CHAPTER FOUR: RESULTS AND DISCUSSION

4.1 Analysis 29

4.2 Testing 29-31

CHAPTER FIVE: CONCLUSION AND RECOMMENDATION

5.1 Conclusion 32

5.2 Recommendation 32

REFERENCES 33-34

APPENDIX A: Source Code for Implemented Model 35-39

LIST OF FIGURES

	Pages
Fig 3.1 Block diagram of the system	17
Fig 3.2 Mounting the antenna	18
Fig 3.3 Plugging the Network cable	18
Fig 3.4: Running the program	19
Fig 3.5 IP Camera Tool Window	19
Fig 3.6 Entering the Username and Password	20
Fig 3.7 Login User Interface	21
Fig 3.8 Installing the ActiveX	21
Fig 3.9 Running the ActiveX	22
Fig 3.10 Live Video Feed	22
Fig 3.11 Flowchart of the system	25
Fig 3.12 Graphical User Interface (GUI) of the system	26
Fig 3.13 Fruit Dataset	27
Fig 4.1 Result showing the detection and classification of an apple	30
Fig 4.2 Result showing the detection and classification of a watermelon	30
Fig 4.3 Result showing the detection and classification of a banana	31
Fig 4.4 Result showing the detection and classification of a lemon	31

CHAPTER ONE

INTRODUCTION

1.1 Background of Study

The essential sources of dietary fiber, vitamins (in particular vitamin C) and antioxidants are fruits. While fresh fruits are subject to spoilage, refrigeration or the removal of oxygen from their storage or packaging containers can extend their shelf life. Dehydration, canning, fermentation, and pickling can turn fruits into juices, jams, and jellies and preserve them.

Due to the growing population, there are rising requirements for high quality fruits. Detection of fruit is a critical activity in the agricultural industry, as is the high demand for good quality fruits on the market. The traditional manual calculation of fruit quality is time-consuming, requiring more manpower and some human error. To minimize human error and increase the speed of the process, some methodologies for automation is introduced.

For years, image recognition and detection has become a prominent machine vision research field and one of the most promising image processing and understanding applications. The image-based recognition system in machine vision is the ability of a system or software to recognize images of objects, people, places and behavior. It utilizes technologies of computer vision with artificial intelligence with and trained algorithms to recognize images via a camera.

A matrix of numerical values is represented by a digital image. These numbers reflect the data associated with each image pixel. Intensities and the position of various pixels in the image are the information fed to the recognition system. With the aid of this data, as part of the learning process, the systems learn to map out a relationship or pattern in the subsequent images given to them.

This work mainly consists of two stages. The textural features are extracted from fruit in the first stage and the fruit is classified as detected fruit in the second stage. The measurements obtained from the study of textural feature are given as input to the classifier for training in order to classify it accordingly. Finally, the system will detect the fruits and will display as an output. The objective of this project is to design a model to recognize the fruits based on size, shape and colour of the fruit while ignoring external features like environment, noise and background.

1.2 Statement of Problem

Detection and classification of fruits in manufacturing and processing industries manually is the main obstacle as it is time consuming, requires more manpower, prone to errors, boring and cost ineffective. Therefore, it is important to develop a reliable mechanism for detection and classification of fruits for various necessities in industries.

1.3 Aim and Objectives

The aim of this project is to develop an image-based system in MATLAB to classify fruits using machine learning algorithm.

The objectives of this project are to:

i. create and train database for a specific variety of fruits
ii. classify images using template matching algorithm
iii. create a database to save the dataset of classified images
iv. design a GUI on MATLAB to display real time feed from IP Camera

1.4 Justification of Study

To enhance the quality and quantity of the agriculture product, there is a need to adopt the new technology. Image recognition approach is a noninvasive technique, which provides consistent, reasonably accurate, less time consuming and cost effective solution**.**

1.5 Scope of Study

The scope of the study is limited to developing an image-based system for classifying fruits in real-time.

1.6 Organization of Report

Chapter one contains background of study, problem statement, aim and objectives, justification of study, scope of study and organization of the report. Chapter two gives a detailed explanation on the fundamentals of image processing in machine vision and related works. The system specifications and the process of data acquisition, preprocessing, feature extraction and training were discussed in Chapter three. Analysis of results and discussion was carried out in Chapter four while the conclusion and direction for future works was presented in Chapter five.

CHAPTER TWO

LITERATURE REVIEW

2.1 Machine Vision

Machine vision, in the agricultural industry, uses existing technologies to classify fruit products before they are packaged to ensure the fruits are correctly labelled. Machine vision automates complex or mundane visual inspection activities and specifically guides handling equipment during product assembly using sensors (cameras), computing hardware, and software algorithms.

Machine vision has advanced in leaps and bounds since the invention of computer vision. If machine vision is considered as the body of a system, then computer vision is the retina, optic nerve, brain, and central nervous system. A machine vision system uses a camera to display an image, which is then processed and interpreted by computer vision algorithms before instructing other components in the system to act. [1]

2.2 Image Processing

We humans can easily recognize and differentiate various features of objects when it comes to recognizing images. This is because our brains have been unconsciously conditioned with the same collection of images, resulting in the creation of abilities to distinguish between objects with ease.

Unlike human brains, computers see visuals as a set of numerical values and search for patterns in digital images, whether still, video, picture, or even live, in order to identify and differentiate key features. A system's interpretation of a picture differs significantly from that of humans. Image processing algorithms are used in computer vision to analyze and recognize visuals from

a single image or a series of images. Computer vision is used to accurately distinguish pedestrians and vehicles on the road by categorizing and filtering millions of user-uploaded images.

The image-based recognition system in machine vision is the ability of a system or software to recognize images of objects, people, places and behavior. [2]

2.3 How Image Processing Works?

A matrix of numerical values is represented by a digital image. These numbers reflect the data associated with each image pixel. The strength of the various pixels averages out to a single value, which is then represented in a matrix format.

The intensities and locations of different pixels in the image are fed to the recognition systems as data. The systems learn to map out a relationship or pattern in the subsequent images supplied to them as part of the learning process using this knowledge.

The system's output on test data is checked after the training phase is completed.

This algorithm takes an image as input and returns the contents of the image. To put it another way, the output is a label. How does an image recognition algorithm figure out what's in a picture? To learn the distinctions between different groups, the algorithm must first be trained.

An image recognition algorithm must be trained with thousands of images of fruits before it is able to identify the fruits. Needless to mention, this algorithm can only comprehend objects/classes that it has previously encountered. [3]

2.4 Overview of Template Matching

Template Matching is a high-level machine vision technique for detecting picture elements that match a predefined template. Template occurrences can be detected using advanced template matching algorithms, regardless of their orientation or local brightness. The Template Matching block finds the best template match inside an input image. The block computes match metric values and then finds the optimal match location by shifting a template over a region of interest or the entire picture. This block outputs the match metric values or the one-based (x,y) coordinates of the best template match. As an alternative, the block can generate an N-by-N matrix of match metric values centered on the best match position.

Input

I is the image. To utilize the template, fill in the blanks with the image you wish to use. The block does not pad the given data. As a result, when the template is centered on the input image, it can only compute values for the match metrics between the input image and the template. The input image's valid region is specified as a collection of all such template locations. The difference in size between the input and template images plus one determines the size of the valid region.

$$\text{Valid size} = \text{Input size} - \text{Template size} + 1$$

T is the template and it can also be specified as an *M*-by-*N* matrix. When utilizing the Template Matching block, the input data is not padded. As a result, when the template is centered on the input image, it can only compute values for the match metrics between the input image and the template. The input image's valid region is specified as a collection of all such template locations. The s is determined by the size difference between the input and template photos plus

one. The difference in size between the input and template images plus one determines the size of the valid region.

$$\text{Valid size} = \text{Input size} - \text{Template size} + 1$$

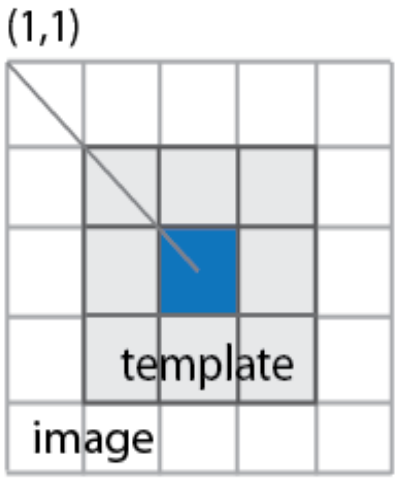
ROI is the Region of Interest and it can be considered a four-element vector. Region of interest vector in the format (*x*,*y*,*width*,*height*), where (*x*,*y*) are one-based coordinates for the upper-left corner of the region The best match position index in reference to the top left corner of the input image is returned by this block.

Output

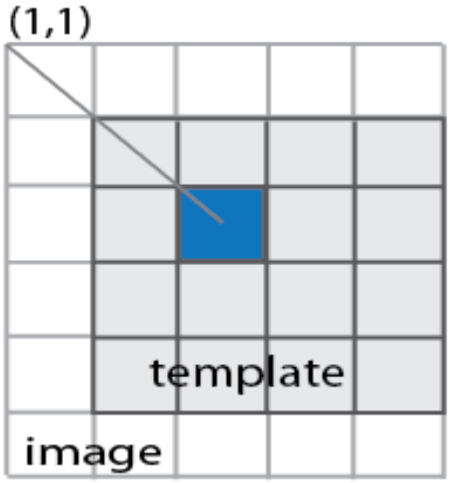
Metric is the Match Metric values. The values of match metrics are arranged in a matrix. Single-step exhaustive window iteration is always implemented in the match metrics matrix. As a result, the block calculates the metric values for each pixel.

Loc is the best match locations and it can be considered a two-element vector. At the one-based (x,y) coordinates, the best template match can be found. When the block is in ROI processing mode, the picture around the ROI is treated as an extension of the ROI subregion. As a result, it calculates the best match sites that are accurate to the ROI's actual borders. The block returns the best match coordinates in relation to the image's top-left corner. The location's [x y] coordinates are one-based and correspond to the template's center. The table below shows how the center coordinates for odd and even templates are output by the block:

Odd number of pixels in template



Even number of pixels in template



NMetric is the Metric values in the neighbourhood of the best match. As an N-by-N matrix, metric values in the vicinity of the best match are returned. N must be an odd number, as indicated in the mask. An exhaustive loop computes all the metric values for the N-by-N neighborhood when you select Best match location to return the matrix of metrics in a neighborhood surrounding the best match.

NValid is the valid neighbourhood which allows you to keep track of the valid neighborhood region. Within the ROI, the neighborhood matrix of metric values is valid. The NValid output is set as follows by the block:

i. 1 — The neighborhood with the best match is entirely inside the interest area.
ii. 0 — The best match's neighborhood is entirely or partially outside of the region of interest.

ROI Valid is the valid region of interest which allows you to track the valid area of interest. The block only analyzes the intersection of the ROI and the valid image if the ROI is partially outside the valid picture. The ROI Valid output is set as follows by the block:

i. 1 — The ROI is totally contained within the input image's valid area.

ii. 0 — The ROI is entirely or partially outside of the input image's valid area.

2.5 Related Works

Deepika Shukla et al (2016) recognized nine different classes of fruits using hybrid features and machine learning. Fruit image dataset were obtained from web as well as certain images were acquired by using mobile phone camera. These images were pre-processed to subtract the background and extract the blob representing fruit. To represent the fruits and capture their visual characteristics, combination of color, shape and texture features were used. These feature datasets were further passed to two different classifiers multiclass SVM and KNN. The color image was firstly converted to grayscale by GLCM (Gray Level Co-occurrence Matrix) then further converted to binary image. Morphological operations were used to fill the holes and extract the largest blob or object from the image which would further be considered as fruit. The largest blob is cropped and the binary values are replaced with original intensity values. From this experiment, it can be concluded that the combination of color, texture and shape gives better results in most of the cases than when any two categories of features are used. Also, that KNN gives better results for this case (91.3%) than SVM (86.96%).

Erwin et al (2017) developed a content-based image retrieval system for multi-objects fruits recognition using k-means and k-nearest neighbor. The system consists of 3 stages. The feature

extraction stage which used Fuzzy Color Histogram (FCH) method for colour extraction and Moment Invariants (MI) method for shape extraction, the clustering stage used K-Means clustering algorithm and the recognition stage used KNN method by calculating the distance between the new fruits image features and features of the existing fruits in the database by using Euclidian distance which then matched with the clustering results. The uniqueness of fruits can be observed using the colors and shapes. Experimental results and analysis of the system obtained an accuracy of 92.5% for single-object images and 90% for the multi-object image.

Yogesh et al (2016) developed a fruit defect detection system based on the use of speeded up robust feature. The local features of the segmented images are extracted and the object recognition was obtained. The basic steps were to create a database of image to be classified, pre-process the image which was done by means of various image processing techniques to improve the quality of the image, apply several filters to de-noise the image and classify the image using image classifiers. The fruit image was converted from RGB image to intensity image and based on the speeded up robust technique, local feature was extracted and described. The feature extraction like shape, size, color and texture of fruits were calculated using algorithm. The matching of disease classification patterns is used. The system also features specialized skin defect identification algorithms that can alter their assignment to a standard category by not only locating them but also determining their distribution.

Mandeep Kaur et al (2015) developed a system using image-processing techniques to classify and grade quality of fruits. A method to increase the accuracy of the fruit quality detection by using artificial neural network [ANN] was used. The first step was to get the image of fruit. Image of the fruit samples were captured by using regular digital camera with white background with the help of a stand. The image of the fruit was put into the Matlab environment, along with feature

extraction for each sample in the dataset for neural network training. The fruit samples' characteristics were extracted and the data was trained using a neural network. A fruit sample was chosen from the database for testing and the testing was performed by using ANN training module button. Finally, ANN based results were obtained and the user has the option to select the sample of fruit which it wants to test and obtain.

Bengi Ozturk et al (2016) developed an algorithm to detect green and orange color fruits in outdoor conditions for robotics applications. Image Segmentation was one of the steps in the process, which means that the original image is divided into many parts. Its goal was to group pixels on the image that have similar characteristics. The goal of detecting fruit was to differentiate the fruits from the background in an image. The k-means algorithm has been chosen as the preferred method. Iteration is required by the K-means method. The data was first clustered according to clustering centers, which were then determined. Pixels chosen at random was used to value the centers. Because there were three dominating hues in the image where the fruits are orange color, the number of cluster sets are 3. The green color ones are clustered 6-8 sets, because the dominant color is green in the image. Filtering operators were given the option of deleting or clearing unnecessary details, depending on the goal. The pixel values were recalculated in this process. When determining round objects, the image was converted to a binary image to make it easier to find them.

Inkyu Sa et al (2016) developed a fruit detection system using deep convolutional neural networks. Two modules were used to complete the task of fruit detection utilizing images: color (RGB) and near-infrared (NIR). Both color and texture were used in the system. The color characteristics were created by translating RGB values to the HSV color space directly. The NIR channel was used to extract visual texture features. NIR images were used to calculate texture

features using (i) Sparse Auto-encoder (SAE) features (ii) Local Binary Pattern (LBP) and (iii) a Histogram of Gradients (HoG). Each feature captured a different attribute, such as the local gradient distribution, edges, and texture, respectively. On large-scale picture categorization and detection, it combined Faster R-CNN with deep convolutional neural networks. The model parameters are updated, or adapted, during fine-tuning using the new data. The categorization decisions from the two models are combined in late fusion. The VGG network was modified and tweaked such that it could receive RGB and NIR data at the same time.

Woo Chaw Seng (2014) developed a new method for fruit recognition system. The K-Nearest Neighbours algorithm was employed as a classifier to classify fruit based on mean color values, shape roundness values, area, and perimeter values. Fruit area and perimeter values were calculated in pixels, but the area of a fruit can be calculated by counting the total number of pixels encircled by the detected fruit boundary area. The user chooses a scalar number to resize the fruit image such that it is roughly the same size as fruit in the real world. The algorithm will compute the new area and perimeter values for the fruit image after picking a desired scalar value. The system is now ready to use after the training data has been entered. The distance between the feature values of the test input fruit and the stored training fruits is determined by the Euclidean distance. The developed recognition system was able to recognize all of the test fruit photos that were selected by the user or system tester from the system's fruit selection menu.

Yudong Zhang and Lenan Wu (2012) presented a multi-class kernel support vector machine-based classification technique (kSVM). A digital camera was used to capture fruit photos, and then a split-and-merge method was used to remove the backdrop of each image. Second, each fruit image's color histogram, texture, and form data were extracted to create a feature space. Third, the dimensions of feature space were reduced using principal component analysis (PCA).

The Image Segmentation was done with Spilt and Merge Algorithm, which is based on a quadtree division of an image and removes the background area. This approach begins from the top of the image's tree, which represents the entire image. If it is discovered to be inhomogeneous, it is divided into four son-squares. On the other hand, if four son-squares are homogeneous, they can be merged into a single component (the merging process). The histogram is then only a representation of the count of pixels that have each possible color.

M. Shamim Hossain et al (2018) proposed a deep learning-based system for fruit classification. The framework was built on top of two separate deep learning architectures. The first was a six-layer convolutional neural network proposed light model, and the second was a fine-tuned visual geometry group-16 pre-trained deep learning model. The proposed system was evaluated using two color image datasets, one of which was publically available. The first dataset (dataset 1) contained obvious clear fruit photos, while the second dataset (dataset 2) contained difficult-to-classify fruit photos. On dataset 1, the first and second models achieved classification accuracy of 99.49 percent and 99.75 percent, respectively. The first and second models had accuracy of 85.43 percent and 96.75 percent, respectively, on dataset 2.

T. Ishikawa et al (2018) proposed a system to classify strawberry fruit shape using machine learning. They introduce the concept of computerization into this sector, as well as the classification. From the data, four categories of descriptors were extracted; the Measured Values (MVs), which include the length of the contour line, the area, the fruit length and width, and the fruit color, the Ellipse Similarity Index (ESI), Elliptic Fourier Descriptors (EFDs), and Chain Code Subtraction (CCS). They also employed these descriptors in conjunction with the random forest methodology for the classification test, and eight of the nine shape types were categorized using combinations of MVs + CCS + EFDs. CCS is a descriptor that adds human understanding

to chain codes, and it outperformed the other descriptors in categorization. The result of this experiment indicates that machine learning has a good ability to classify fruits.

Adeniji Kehinde et al (2019) proposed that on the Principal Count Analysis (PCA) algorithm for facial recognition optimization, it is possible to incorporate Internet of Things experience. The PCA method was used to create an internet-based real-time facial recognition system that can detect and track faces. A label name that denotes the face found was also integrated to make it easier to follow the face in circumstances where several faces are identified at the same time. The IP-camera firmware was used to establish up a wireless connection with an internet protocol (IP)-camera via a router. As the router broadcasts this connection, a connection is established with a personal computer via the network and sharing center tab on the computer system, resulting in a wireless connection between the computer and the internet camera. The video from the camera was accessed using a graphical interface created in MATLAB, and the PCA technique was used to recognise and track faces in real-time. The system built included security measures such as a timestamp and a database.

CHAPTER THREE

METHODOLOGY

3.1 System Requirements

The following are the hardware and software components required to develop the application as well as the functional/non-functional requirements of the system.

3.1.1 Hardware Requirements

i. Laptop

The laptop was used to interface the MATLAB R2018a software with the hardware. A PC with an Intel(R) Core(TM) i7-3520M processor which operates at 2.90GHz, memory of 8GB RAM which provided a platform for the system and application software was used for the implementation of the various models.

ii. IP Camera

An IP Camera is a type of digital video camera commonly employed for surveillance to send and receive data via a computer network and internet. The IP Camera used for this project is a FOSCAM Indoor Pan/Tilt Wireless IP Camera. The model of the camera is FI8910W.

iii. Wireless Access Point

The router was used to connect the IP Camera wirelessly to the MATLAB software. The specifications are 300Mbps, 2.4Ghz - 5.0Ghz, Wireless, 4 antennas.

3.1.2 Software Requirements

The process was implemented on MATLAB R2018a. MATLAB (Matrix Laboratory) is a numerical computing environment and fourth generation computer programming language, a programming language developed by The MathWorks Inc.

MATLAB was used in this project to create a GUI (Graphical User Interface) that interacts with the IP Camera and the system user.

3.1.3 Functional and Non-Functional Requirements

The functional requirements refers to the features that the system possesses and needs to deliver, while the non-functional requirements shows how the system should behave.

The functional requirements are;

- The system includes a real-time fruits detection algorithm
- The system also include a fruit classification algorithm
- The system contains an algorithm for extraction of meaningful fruit features
- The system has a simple GUI

The non-functional requirements are;

- The system should be implemented on MATLAB
- The systems GUI must be simple, descriptive and functional

3.2 System Design

In this work, a router is used to set up a wireless connection with an IP camera. A Graphical User Interface is designed on MATLAB to access live videos and images of fruits via the IP camera that the machine learning algorithm will use to detect and classify the fruits as shown on the IP Camera.

3.2.1 Block Diagram of the system

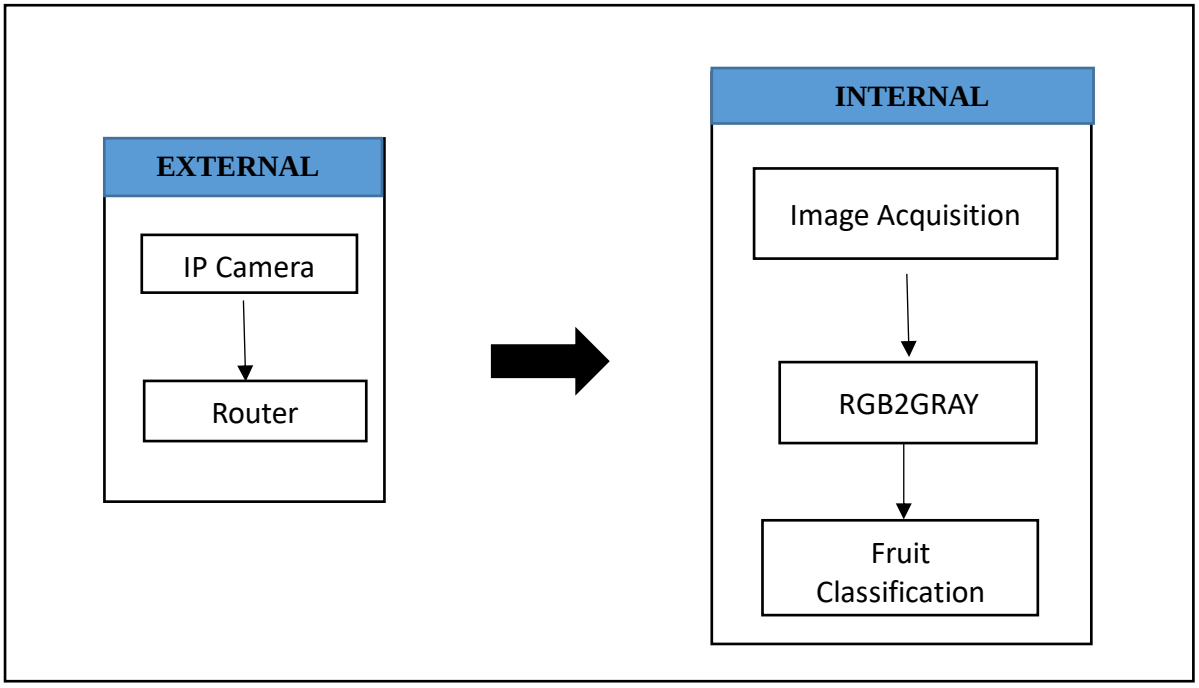


Fig 3.1: Block Diagram of the system

Connecting the IP camera to the router

- **Hardware Installation**

1. Open the box and take out the camera carefully.
2. Take the Wi-Fi antenna, mount it on the SMA connector on the back of the camera, screw the antenna to the bottom, and make the antenna stand vertically.

Fig 3.2 Mounting the Antenna

3. Use the network cable to connect the camera to the router or the switch in the LAN network at your home or your office. Plug in the power. The green network light at the rear of the camera will blink and the camera will automatically pan/tilt. The red power light will also turn on.

Fig 3.3 Plugging the Ethernet cable

- **Software Installation**

1. Insert the CD into the CD drive of your computer and go to the folder “FI8910W”, find the folder “For Windows OS”. Copy the IP camera tool to your computer and start the program.

Fig 3.4 Starting the program

2. Double click the IP Camera Tool icon. The IP camera tool should find the camera’s IP automatically after you plug in the network cable.

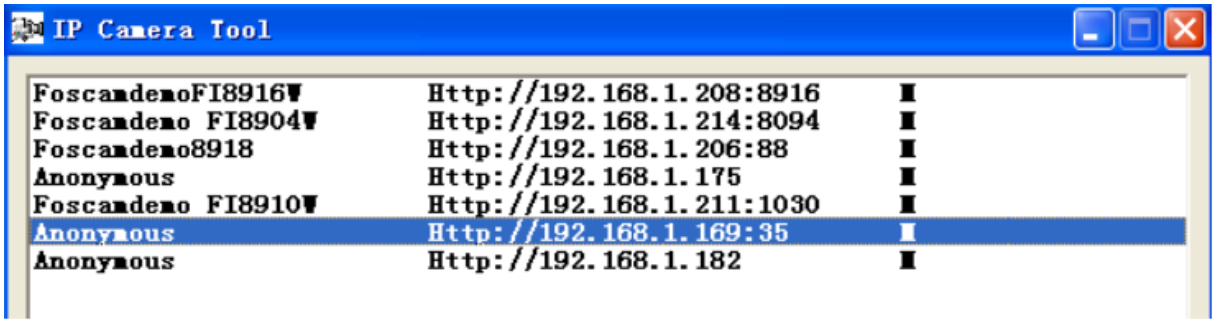


Fig 3.5 IP Camera Tool Window

3. Double click the LAN IP address of the camera, it pops up a password dialog box. Enter the username and password.

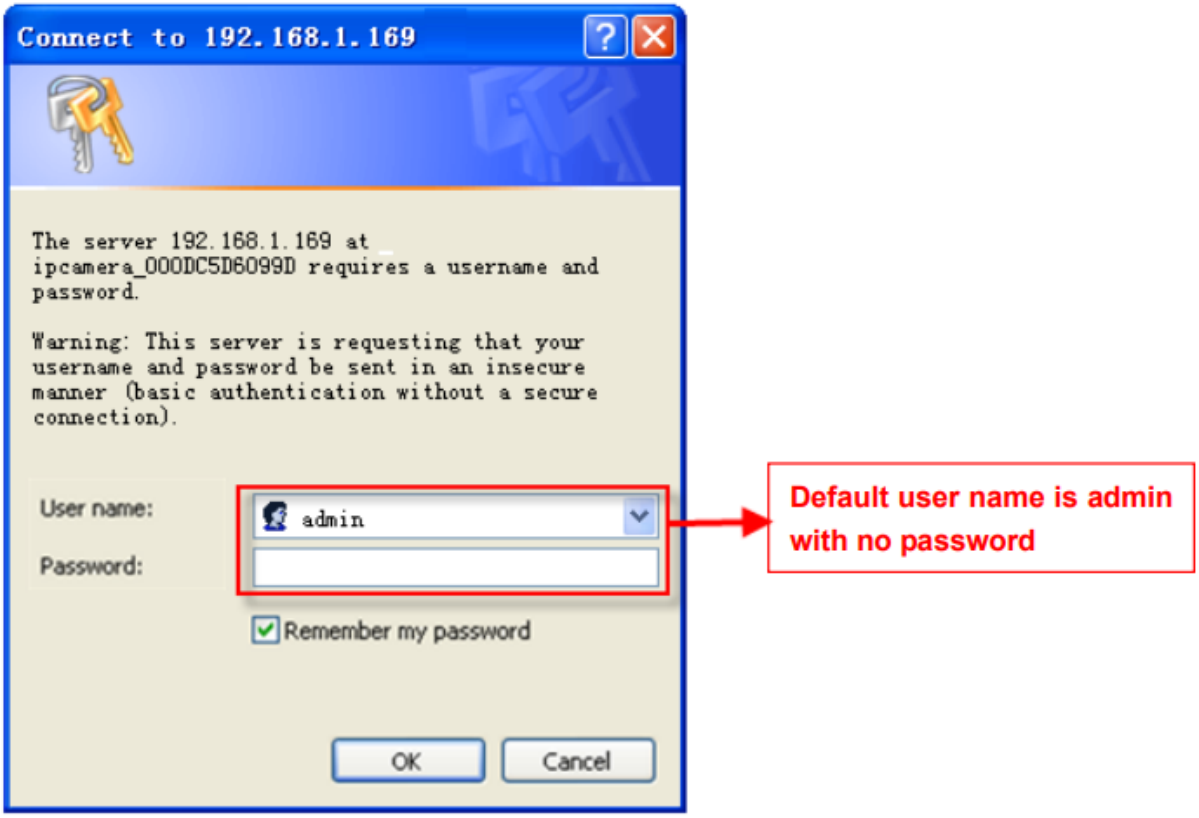


Fig 3.6 Entering the Username and Password

4. There are three login methods. One is IE ActiveX Mode for Internet Explorer, the other is Server Push Mode for Safari, Firefox, Google Chrome, the third mode is Mobile Phone for mobile phone. For the first time logging in the camera, be sure to install ActiveX when prompted, run the ActiveX and re-login to the camera.

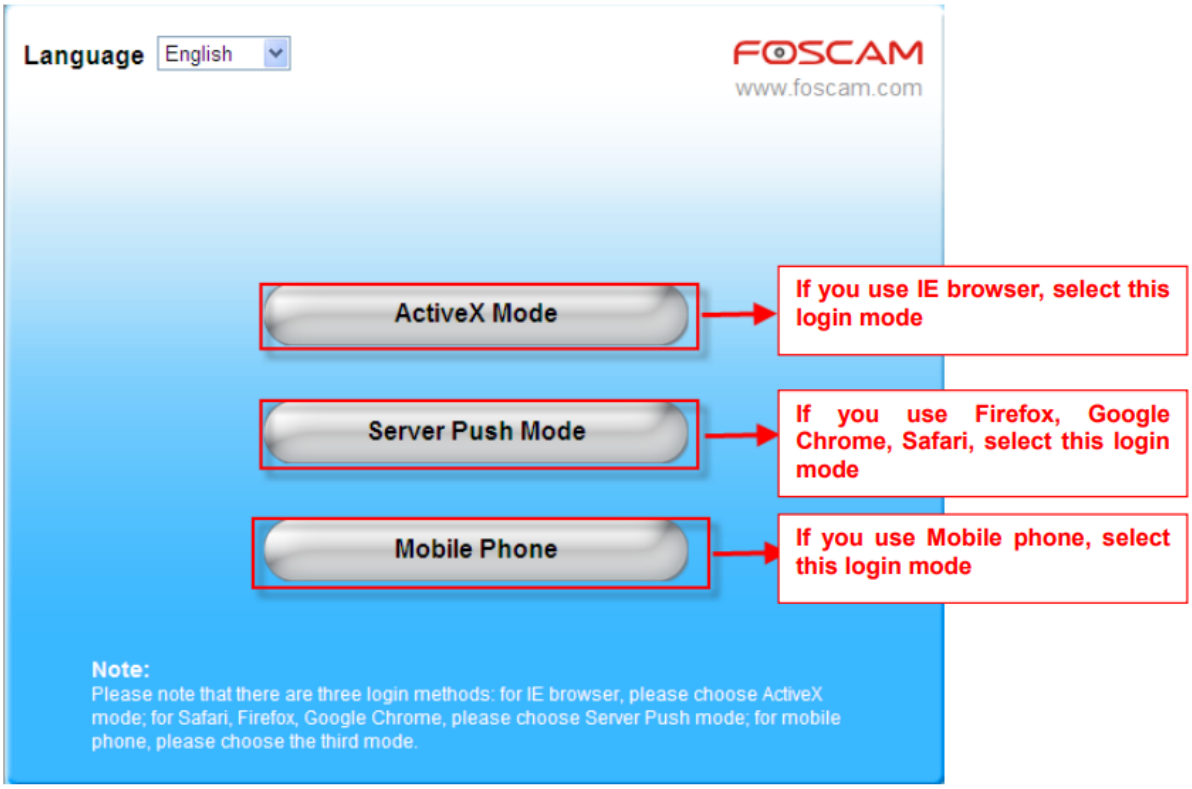


Fig 3.7 Login User Interface

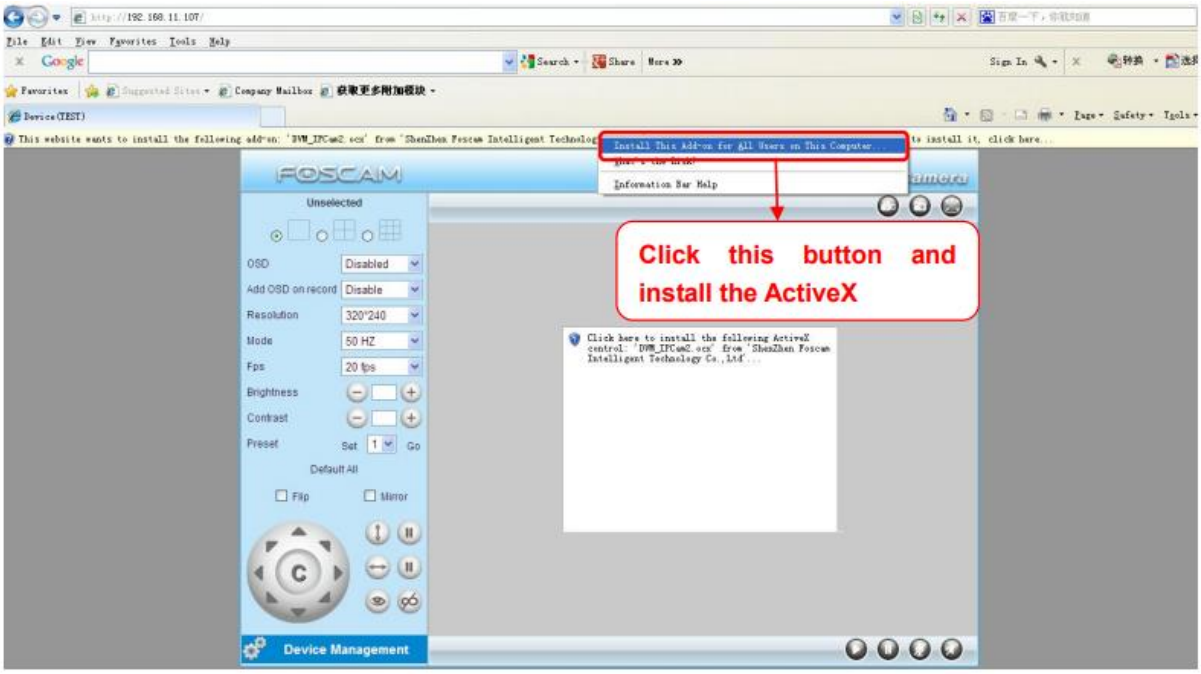


Fig 3.8 Installing the ActiveX

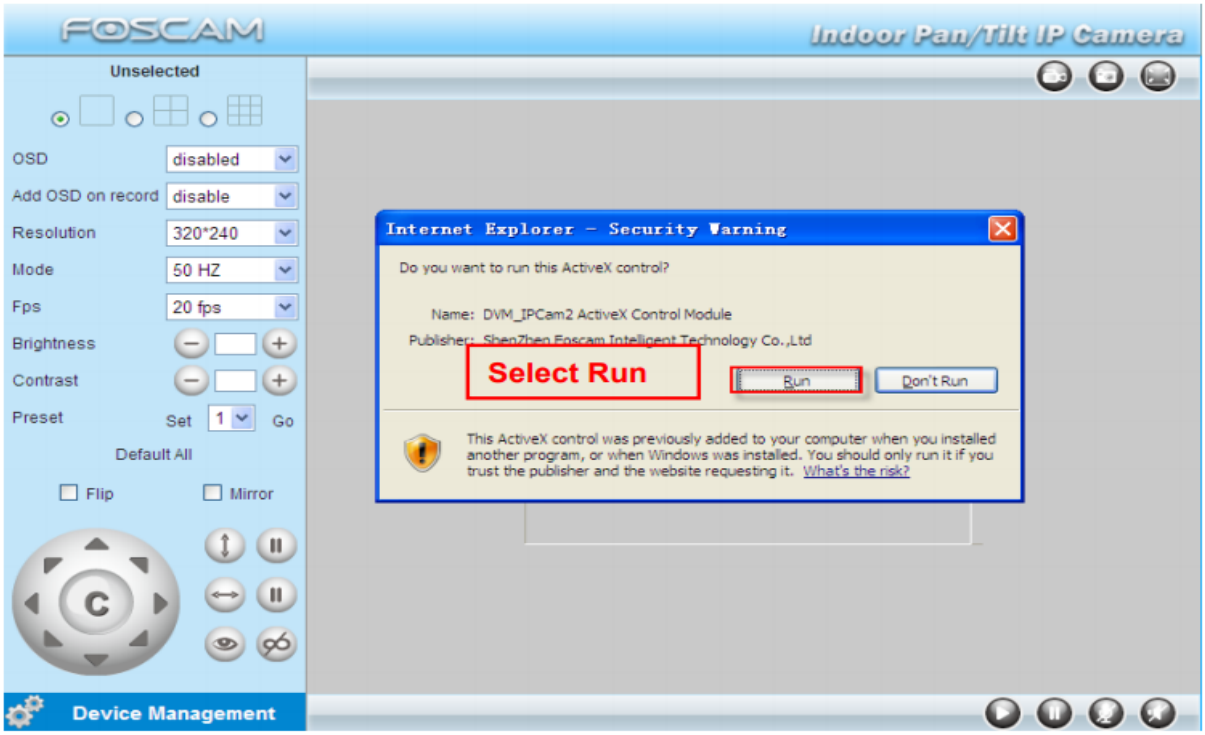


Fig 3.9 Running the ActiveX

Fig 3.10 Live Video Feed

- **Wireless Connection Settings**

1. “Device Management” >> “Wireless LAN Settings” >>“Using Wireless LAN”. Click the Scan button and the camera will detect all WIFI devices around the area including your router in the list
2. Click the SSID of your router in the list, the corresponding information (SSID & Encryption) will be filled in the following boxes automatically. Only the share key needs to be filled in. Make sure that SSID, Encryption and share key you filled in for the camera are exactly the same for your router.
3. Click on the Submit button after all settings have been entered, the camera will reboot. After the camera has completed the reboot process, wait 10 seconds and disconnect the network cable. The LAN IP address will disappear on the window of IP Camera Tool when the camera gets restarted. Just wait for around 1 minute, the camera will get wireless connection, and the LAN IP of the camera will be showed again on the window of the IP Cam Tool. If the camera has a dynamic IP, after the wireless settings, the IP will be changed.

Image Acquisition

An image of a fruit is acquired via the IP Camera and it is loaded and saved using the MATLAB software. The image acquisition technology determines the average image quality the fruit recognition algorithm has to work on i.e. the better the quality of the input images, the better the conditions the fruit classification algorithm has, and thus a higher classification accuracy can achieved.

Rgb2gray

RGB is one of the color picture formats. In this case, the input image is represented by three size matrices based on the picture format. In each image, there are three matrices. The three matrices in each image relates to the colors red, green, and blue, as well as saying how much of each of these colors a given pixel should take advantage of.

Gray scale images have a single color, which is a shade of gray in a range of values. Monochrome image is another name for gray colour image. This indicates that there is a presence of only one color (monochrome). Any color can be converted. To convert a colour image to a grayscale representation of its brightness, the values of its red, green, and blue (RGB) primaries must first be extracted in linear intensity encoding using gamma expansion.

Fruit Classification

Identifying features is the core of fruit classification algorithm. It is possible to estimate a transformation model from features of fruits in images that can be linked to their counterparts characteristics in the previous images. All of the templates that are preloaded into the system are correlated with the input image. When the correlation is finished, the template with the highest associated value is designated as the fruit's classification.

3.2.2 Flowchart of the system

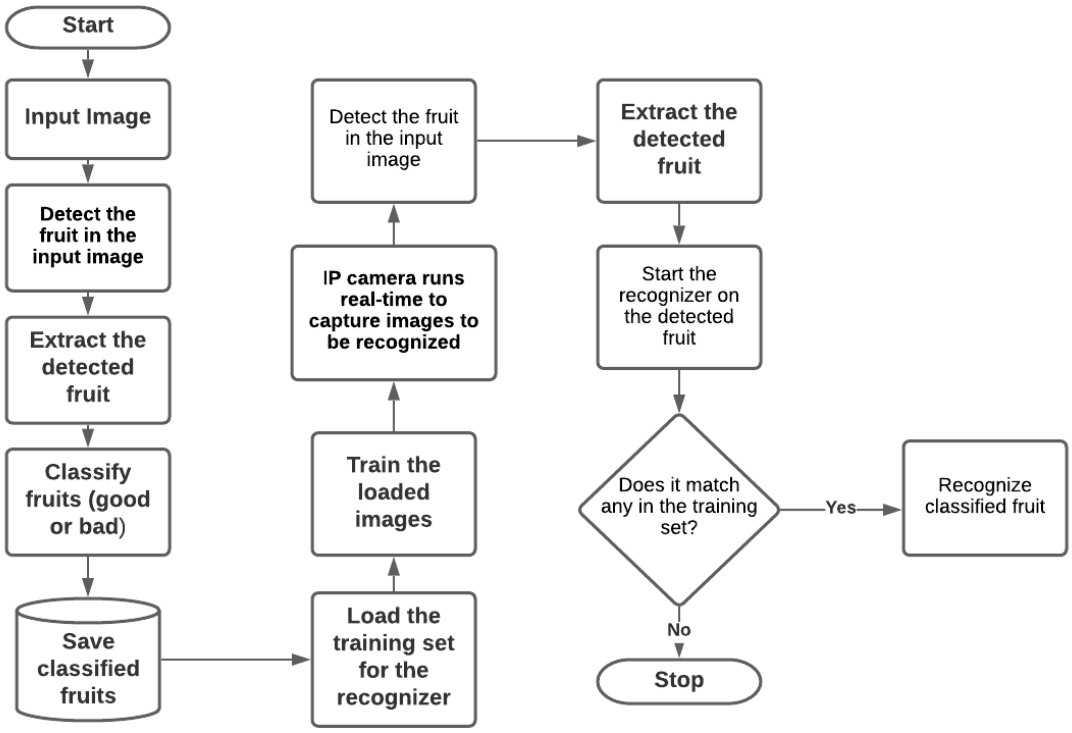


Fig 3.11 Flowchart of the system

The system input is an image of a fruit, the features of the detected fruit is extracted, classified and saved to a database. The training set is loaded for fruit recognition and the recognized fruit is trained.

IP-camera runs in real-time to capture images of fruits to be recognized. The image is detected and the features of the detected fruit is extracted and recognized. If the detected fruit matches any in the training set, then the fruit is recognized and classified as the fruit in the input image.

3.2.3 Graphical User Interface

Fig 3.12 Graphical User Interface (GUI) of the system

The GUI of this system consists of an axes in which the image captured by the IP camera is displayed, a 'Start Camera' and 'Stop Camera' pushbutton to control the operation of the IP camera, two edit text tabs; one to display the timestamp and the other to display the result of the classified fruit.

3.3 Data Processing

This aspect is divided into the data collection and the data pre-processing in order to convert the images into a form the algorithm will understand.

3.3.1 Data Collection

For the fruit classification system, nearly thirty (30) images of various fruits were captured using an iphone and android camera and some were gotten from the web. The images were captured using a high resolution camera to ensure better image quality. These images were used

as input to the fruit classification system. The use of a camera with more definition and resolution was used in order to increase the success ratio of the system. Among the captured images, only the clearly visible images were used as the primary data for training the system to ensure we obtain an accurate result.

Fig 3.13 Fruit Dataset

3.3.2 Data Pre-processing

The fruit images were cropped from the main image first and then resized. In order to remove the noise or other unwanted distortion, the images were grey scaled. Also, to reduce the channel of the image from a 3-channel (RGB) image to a 1-channel image to avoid very large images, the images were gray scaled. The new image gotten was used as the input into the model. The processed images were converted to datasets.

3.4 Implementation of the algorithm

Template matching is a digital image processing approach that identifies the parts of a figure that match a predetermined template. It is a technology for identifying small components of a figure that match a template image. When a template is a sub image that contains the form we want to fill out, we tend to match it to an image.

The following are two key sections:

(a) Source Image (I): This is the image inside which we are looking for a match to the template image.

(b) Template Image (T): The path image that can be compared to the template image, with the goal of determining the most effective strategy for best region matching. The matching technique computes the inaccuracy among images in addition to taking the similarity measure.

The correlation coefficient is calculated using the MATLAB program ‘corr2'. To create the correlation map, the Target Image is put over the template image, and the correlation coefficient for each pixel in the template image is found. The greatest coefficient is determined from the map after sliding through all of the pixels in the template image. The starting point of the target image is the pixel position with the highest value.

```
new_x=corr2(x,currentimage);    (1)
```

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 Analysis

An input image is read into the system via an IP camera. Prior to this, a dataset of three fruits (apple, banana and watermelon) are collected, trained, and tested through a template matching algorithm which is a high-level machine vision technique for detecting elements of an image that match a predetermined template. The correlation coefficient is calculated using the MATLAB program ‘corr2'. The Target Image is put over the template image, and the correlation coefficient for each pixel in the template image is found. The greatest coefficient is determined from the map after sliding through all of the pixels in the template image.

The snapshot taken by the IP camera is automatically saved in the same directory as the dataset of the fruits previously taken and through the help of template matching, an accurate classification result is arrived at. If a new image that doesn’t match any in the dataset is taken, the system would not be able to recognize and classify it and an error message will be displayed.

4.2 Testing

This occurs when a program is run with the goal of detecting faults. A software must be error-free in order to perform well. If the testing is completed successfully, the software will be free of all errors. All tests must satisfy the customer's needs.

The testing approach used was integration testing. The goal is to take unit-tested components and use them to create a program structure that is dictated by design. Integration testing entails combining a number of components to achieve a result.

When the input image is an apple;

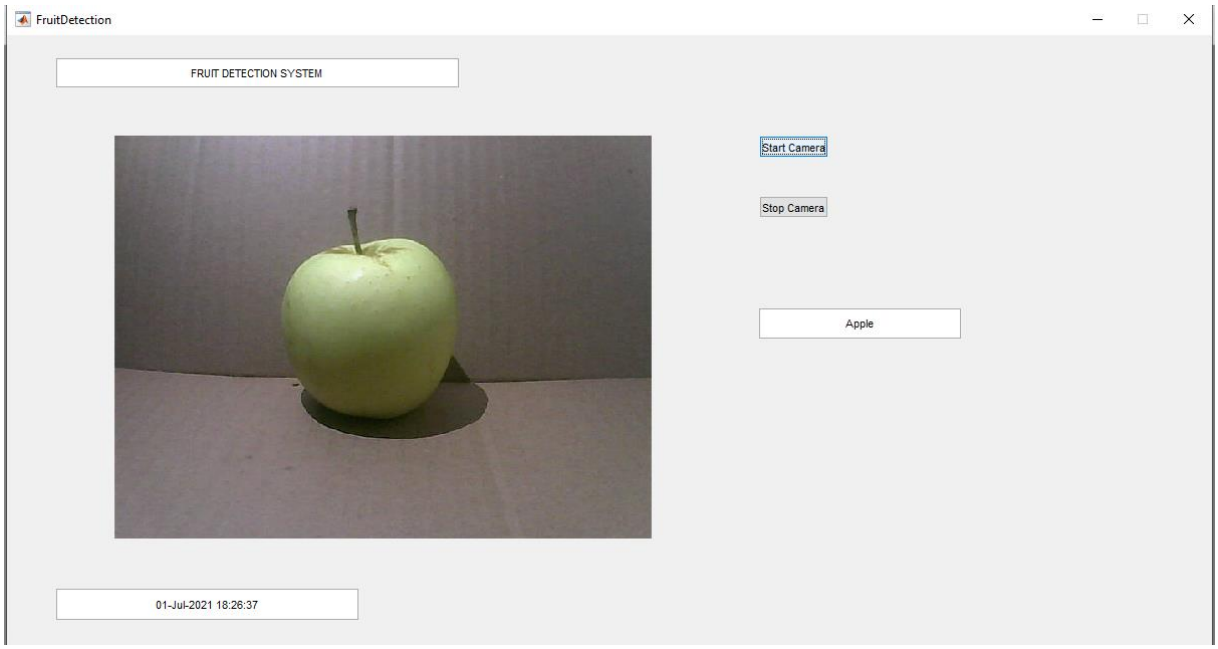


Fig 4.1 Result showing the detection and classification of an apple

When the input image is a watermelon;

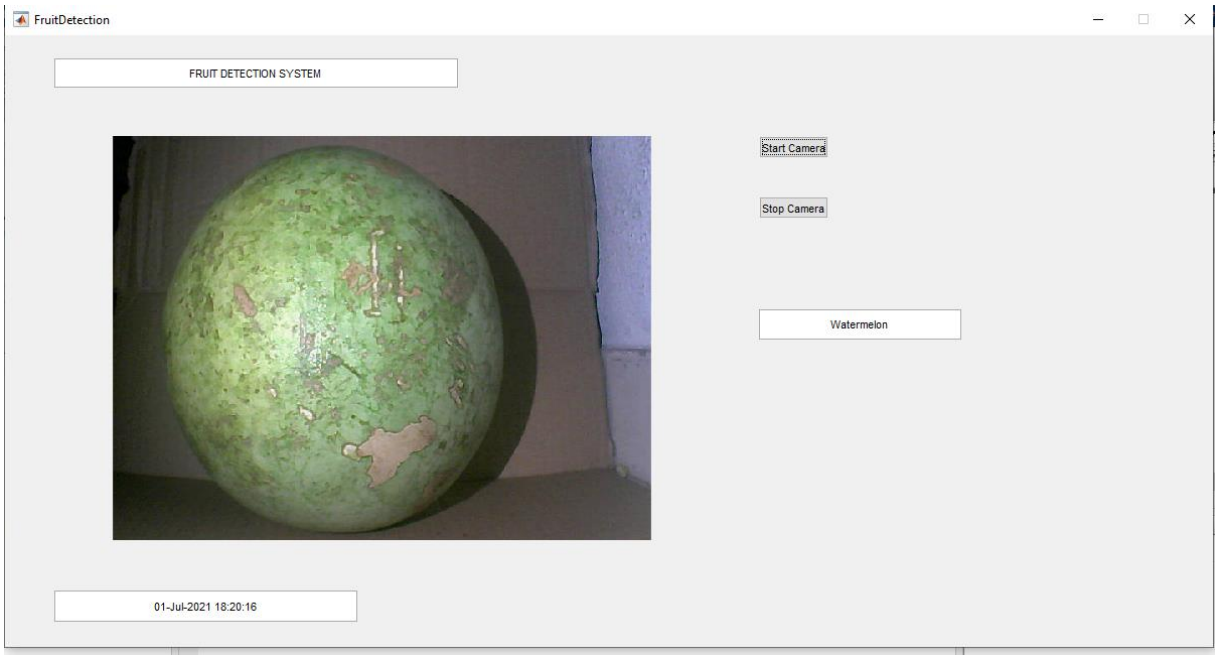


Fig 4.2 Result showing the detection and classification of a watermelon

When the input image is a banana;

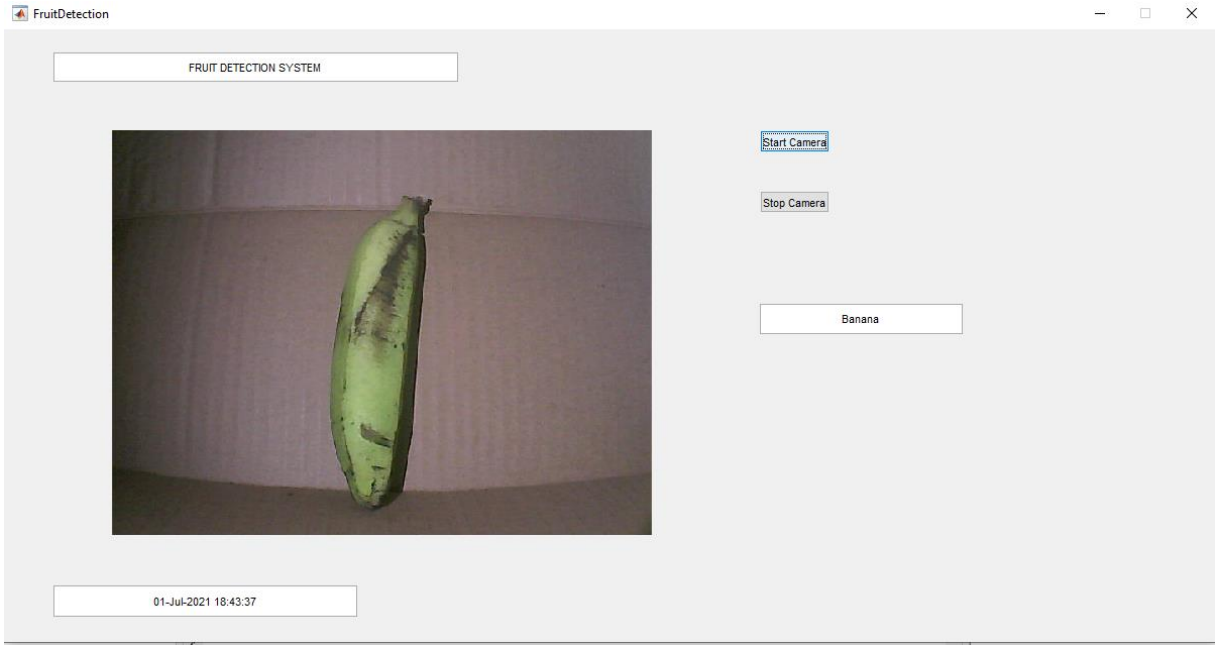


Fig 4.3 Result showing the detection and classification of a banana

Otherwise,

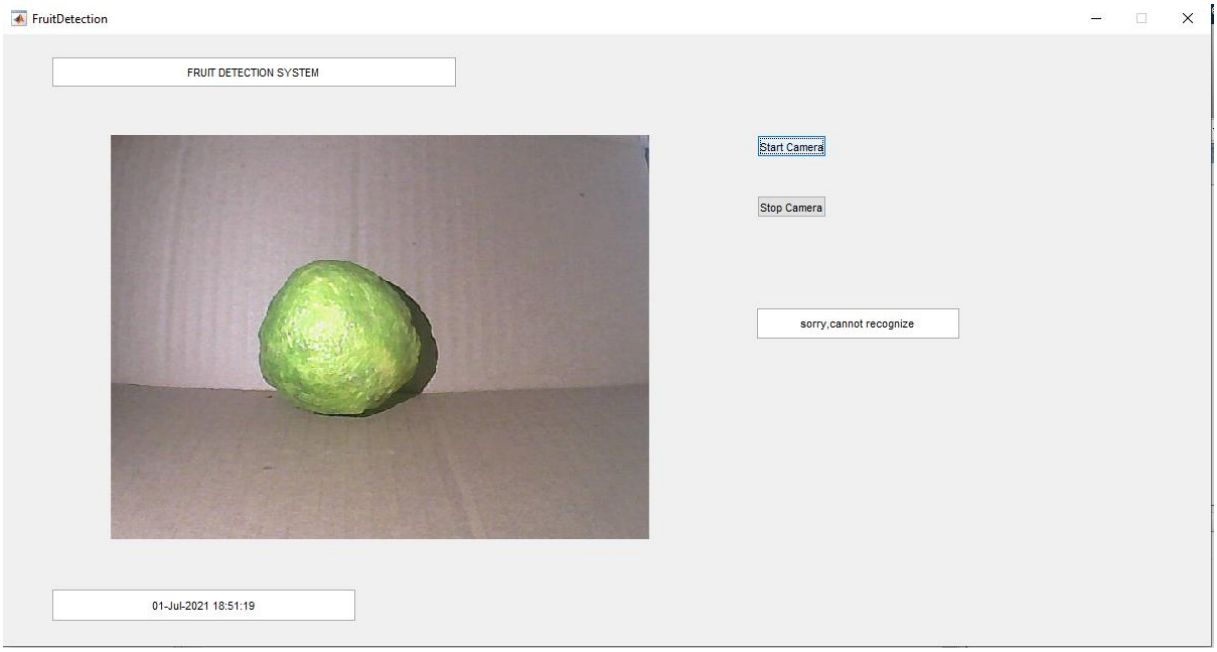


Fig 4.4 Result showing the detection and classification of a lemon

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

Image processing algorithm is the basis for most recognition task ranging from classification, object-detection to segmentation. In this project, an image based GUI system for real time detection and classification of fruits was carried out. After a couple of algorithms were implemented for fruit recognition, thorough evaluation was carried out to obtain an algorithm with a better performance and a template matching algorithm was the best fit.

The algorithm was tested for four different kinds of food, three of which were a dataset in the system and implemented using MATLAB. The system developed is a general system which works well with any type of image and has no restrictions on the size of an image. The snapshot taken via the IP camera is saved as a PNG file.

The template matching approach provides a practical solution that Is well suited to the problem of fruit detection and classification. It is a secure, relatively simple and effective system that has been developed to replace manual systems.

5.2 Recommendations

Because the dataset used was relatively small, the classification result was affected mildly. It is recommended that a larger and expanded dataset should be used as it will improve the overall performance of the fruit detection system.

The result of the classified fruits can also be sent to the cloud for future reference and easy access by a manager.

References

1. Teledynedalsa.com. 2021. *Machine Vision 101: An Introduction | Teledyne DALSA*. [online] Available at: <https://www.teledynedalsa.com/en/learn/knowledge-center/machine-vision-101-an-introduction/> [Accessed 31 May 2021].

2. Learn OpenCV | OpenCV, PyTorch, Keras, Tensorflow examples and tutorials. 2021. *Image Recognition and Object Detection : Part 1 | Learn OpenCV*. [online] Available at: <https://learnopencv.com/image-recognition-and-object-detection-part1/> [Accessed 31 May 2021].

3. Einfochips.com. 2021. *Understanding Image Recognition and Its Uses*. [online] Available at: <https://www.einfochips.com/blog/understanding-image-recognition-and-its-uses/> [Accessed 31 May 2021].

4. Mathworks.com. 2021. *Locate template in image - MATLAB*. [online] Available at: <https://www.mathworks.com/help/vision/ref/vision.templatematcher-system-object.html> [Accessed 1 July 2021].

5. Shukla, D., 2016. *Recognition of fruits using hybrid features and machine learning*. [online] Ieeexplore.ieee.org. Available at: <https://ieeexplore.ieee.org/document/7915033> [Accessed 28 June 2021].

6. Erwin., 2017. *Content based image retrieval for multi-objects fruits recognition using k-means and k-nearest neighbor*. [online] Ieeexplore.ieee.org. Available at: <https://ieeexplore.ieee.org/document/8285855> [Accessed 28 June 2021].

7. Yogesh., 2016. *Fruit defect detection based on speeded up robust feature technique*. [online] Ieeexplore.ieee.org. Available at: <https://ieeexplore.ieee.org/document/7785023> [Accessed 28 June 2021].

8. Hossain, M. and Al-Hammadi, M., 2018. *Automatic Fruit Classification Using Deep Learning for Industrial Applications*. [online] Ieeexplore.ieee.org. Available at: <https://ieeexplore.ieee.org/abstract/document/8488544> [Accessed 31 May 2021].

9. Kaur, M., 2015. *(PDF) Quality Detection of Fruits by Using ANN Technique*. [online] ResearchGate. Available at: <https://www.researchgate.net/publication/286933975_Quality_Detection_of_Fruits_by_Using_ANN_Technique> [Accessed 28 June 2021].

10. Ozturk, B., 2016. *Detection of green and orange color fruits in outdoor conditions for robotic applications*. [online] Ieeexplore.ieee.org. Available at: <https://ieeexplore.ieee.org/document/7577641> [Accessed 28 June 2021].

11. Sa, I., 2016. *(PDF) DeepFruits: A Fruit Detection System Using Deep Neural Networks*. [online] ResearchGate. Available at: <https://www.researchgate.net/publication/305824563_DeepFruits_A_Fruit_Detection_System_Using_Deep_Neural_Networks> [Accessed 28 June 2021].

12. Seng, W., 2014. *A new method for fruits recognition system*. [online] Ieeexplore.ieee.org. Available at: <https://ieeexplore.ieee.org/document/5254804> [Accessed 28 June 2021].

13. Zhang, Y., 2014. *(PDF) Classification of Fruits Using Computer Vision and a Multiclass Support Vector Machine*. [online] ResearchGate. Available at: <https://www.researchgate.net/publication/232742471_Classification_of_Fruits_Using_Computer_Vision_and_a_Multiclass_Support_Vector_Machine> [Accessed 28 June 2021].

14. Ishikawa, T., Hayashi, A., Nagamatsu, S., Kyutoku, Y., Dan, I., Wada, T., Oku, K., Saeki, Y., Uto, T., Tanabata, T., Isobe, S. and Kochi, N., 2018. CLASSIFICATION OF STRAWBERRY FRUIT SHAPE BY MACHINE LEARNING. *ISPRS - International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLII-2, pp.463-470. [Accessed 31 May 2021].

15. Kehinde, A., 2021. *Conscientia Beam*. [online] Conscientiabeam.com. Available at: <http://www.conscientiabeam.com/journal/76/html/5703> [Accessed 8 July 2021].

16. GeeksforGeeks. 2021. *Types of Software Testing - GeeksforGeeks*. [online] Available at: <https://www.geeksforgeeks.org/types-software-testing/> [Accessed 8 July 2021].

Appendix

```
function varargout = FruitDetection(varargin)
% FRUITDETECTION MATLAB code for FruitDetection.fig
%      FRUITDETECTION, by itself, creates a new FRUITDETECTION or raises
the existing
%      singleton*.
%
%      H = FRUITDETECTION returns the handle to a new FRUITDETECTION or the
handle to
%      the existing singleton*.
%
%      FRUITDETECTION('CALLBACK',hObject,eventData,handles,...) calls the
local
%      function named CALLBACK in FRUITDETECTION.M with the given input
arguments.
%
%      FRUITDETECTION('Property','Value',...) creates a new FRUITDETECTION
or raises the
%      existing singleton*.  Starting from the left, property value pairs
are
%      applied to the GUI before FruitDetection_OpeningFcn gets called.  An
%      unrecognized property name or invalid value makes property
application
%      stop.  All inputs are passed to FruitDetection_OpeningFcn via
varargin.
%
%      *See GUI Options on GUIDE's Tools menu.  Choose "GUI allows only one
%      instance to run (singleton)".
%
% See also: GUIDE, GUIDATA, GUIHANDLES

% Edit the above text to modify the response to help FruitDetection

% Last Modified by GUIDE v2.5 25-Jun-2021 14:50:45

% Begin initialization code - DO NOT EDIT
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                   'gui_Singleton',  gui_Singleton, ...
                   'gui_OpeningFcn', @FruitDetection_OpeningFcn, ...
                   'gui_OutputFcn',  @FruitDetection_OutputFcn, ...
                   'gui_LayoutFcn',  [] , ...
                   'gui_Callback',   []);
if nargin && ischar(varargin{1})
    gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
    [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
    gui_mainfcn(gui_State, varargin{:});
end
% End initialization code - DO NOT EDIT


% --- Executes just before FruitDetection is made visible.
function FruitDetection_OpeningFcn(hObject, eventdata, handles, varargin)
% This function has no output args, see OutputFcn.
% hObject    handle to figure
```

```
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
% varargin   command line arguments to FruitDetection (see VARARGIN)

% Choose default command line output for FruitDetection
handles.output = hObject;

% Update handles structure
guidata(hObject, handles);

% UIWAIT makes FruitDetection wait for user response (see UIRESUME)
% uiwait(handles.figure1);


% --- Outputs from this function are returned to the command line.
function varargout = FruitDetection_OutputFcn(hObject, eventdata, handles)
% varargout  cell array for returning output args (see VARARGOUT);
% hObject    handle to figure
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Get default command line output from handles structure
varargout{1} = handles.output;



function timestamp_Callback(hObject, eventdata, handles)
% hObject    handle to timestamp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of timestamp as text
%        str2double(get(hObject,'String')) returns contents of timestamp as
a double


% --- Executes during object creation, after setting all properties.
function timestamp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to timestamp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end



function edit2_Callback(hObject, eventdata, handles)
% hObject    handle to edit2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit2 as text
```

```
%        str2double(get(hObject,'String')) returns contents of edit2 as a
double

% --- Executes during object creation, after setting all properties.
function edit2_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end


% --- Executes on button press in StartCamera.
function StartCamera_Callback(hObject, eventdata, handles)
% hObject    handle to StartCamera (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

datestr(clock); // convert date and time to string format
cam = ipcam('http://192.168.31.152/video.cgi','admin',''); // call the IP
camera

    a = datestr(clock); //assign a variable to the timestamp function
    set(handles.timestamp, 'String', a); //assign the value of timestamp to
the edit text handle created for the timestamp

    a = snapshot(cam); // take a snapshot
    axes(handles.axes1); // display snapshot on the axes handle
    x = imshow(a); // preview snapshot
   ImageFolder = 'C:\Users\AdminUser\Documents\PROJECT WORK\FRUIT
DETECTION\template\TestImages'; // call the directory you want to save
snapshot to
   for i = 1
       x = snapshot(cam); // take a snapshot
       file_name = sprintf('snapshot.png', i);
       fullFileName = fullfile(ImageFolder,file_name);
       imwrite(x,file_name,'png'); // writes the snapshot taken to the file
specified by filename and save as a png file
       pause(1);
       imshow(x); //preview snapshot
       imgName = [ImageFolder, '\Image_',num2str(i),'.png']; // generate
the name the snapshot image will be saved as
       imwrite(x,imgName); // write image name and image to the specified
directory
   end


if(size(x,3)>1)
    x=rgb2gray(x);
end
x=double(x);
[m,n]=size(x);
x=imresize(x,[m,m]);
new_x=zeros(m,n);
```

```
% % %Code for Banana
imagefiles_banana = dir('template\bannana\*.*');
nfilesbanana = length(imagefiles_banana);
outputMatchingResultbanana=zeros(2,nfilesbanana);
valu=[];idx=[];
for ii=3:nfilesbanana
    currentfilename =
fullfile(imagefiles_banana(ii).folder,imagefiles_banana(ii).name);
    currentimage = imread(currentfilename);
    if(size(currentimage,3)>1)
        currentimage=rgb2gray(currentimage);
    end
    [p,q]=size(currentimage);
    currentimage=imresize(currentimage,[m,m]);
    currentimage=double(currentimage);
    images{ii} = currentimage;
     new_x=corr2(x,currentimage);
    [valu(ii), idx(ii)]=max(max(new_x));

end
%
%
% %
% % % code for apple

imagefiles_apple = dir('template\apples\*.*');
nfilesapple = length(imagefiles_apple);
outputMatchingResultapple=zeros(2,nfilesapple);
for ii=3:nfilesapple
    currentfilename =
fullfile(imagefiles_apple(ii).folder,imagefiles_apple(ii).name);
    currentimage = imread(currentfilename);
    if(size(currentimage,3)>1)
        currentimage=rgb2gray(currentimage);
    end
    [p,q]=size(currentimage);
    currentimage=imresize(currentimage,[m,m]);
    currentimage=double(currentimage);
    images{ii} = currentimage;
    new_x=corr2(x,currentimage);
    [valu1(ii), idx1(ii)]=max(max(new_x));
end
% %
% % % code for watermelon
%
imagefiles_watermelon = dir('template\watermelon\*.*');
nfileswatermelon = length(imagefiles_watermelon);
outputMatchingResultwatermelon=zeros(2,nfileswatermelon);
for ii=3:nfileswatermelon
    currentfilename =
fullfile(imagefiles_watermelon(ii).folder,imagefiles_watermelon(ii).name);
    currentimage = imread(currentfilename);
    if(size(currentimage,3)>1)
        currentimage=rgb2gray(currentimage);
    end
    [p,q]=size(currentimage);
    currentimage=imresize(currentimage,[m,m]);
    currentimage=double(currentimage);
    images{ii} = currentimage;
    new_x=corr2(x,currentimage);
```

```
    [valu2(ii), idx2(ii)]=max(max(new_x));
end


banana=max(valu);
apple=max(valu1);
watermelon=max(valu2);


if (watermelon>apple) && (watermelon>banana)
    output='Watermelon';
elseif (banana>apple) &&(banana>watermelon)
    output='Banana';
elseif (apple>banana) && (apple>watermelon)
    output='Apple';
else
    output='sorry,cannot recognize';
end

set(handles.edit2,'string',output);
% --- Executes on button press in StopCamera.
function StopCamera_Callback(hObject, eventdata, handles)
% hObject    handle to StopCamera (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
cla(handles.StopCamera,'reset');


function edit3_Callback(hObject, eventdata, handles)
% hObject    handle to edit3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit3 as text
%        str2double(get(hObject,'String')) returns contents of edit3 as a
double


% --- Executes during object creation, after setting all properties.
function edit3_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit3 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc && isequal(get(hObject,'BackgroundColor'),
get(0,'defaultUicontrolBackgroundColor'))
    set(hObject,'BackgroundColor','white');
end
```